

Assessment Task 1 — Professional Learning Plan & Portfolio

Dream Job: Platform Engineer — cloud-native infrastructure

Student name	Jinkyung Kim
Student ID	14657314
Subject	43030 Professional Practice — Spring 2026
Assessment task	Assessment Task 1: Professional Learning Plan & Portfolio (10%)
Target career	Platform Engineer / Cloud Infrastructure Engineer
Date submitted	17 August 2026

What is in this submission. Part A has the two job advertisements and my reasons for picking them. Part B is the skills audit. Part C is the learning plan itself, four goals, and it is the part held to the four-page limit. Where I give a skill level I am using SFIA 9, whose levels of responsibility run 1 Follow, 2 Assist, 3 Apply, 4 Enable, 5 Ensure/Advise, 6 Initiate/Influence, 7 Set strategy.

Part A — Job Advertisements and Selection Rationale

Job Advertisement 1 — Senior Platform Engineer, Kongsberg Digital (Falkor)

Field	Detail
Position	Senior Platform Engineer
Employer	Kongsberg Digital (Falkor) — industrial software, Kongsberg Group
Location	Fornebu, Norway — hybrid, minimum three days per week on site
Employment type	Full-time, permanent; senior level
Source	The Hub (thehub.io/jobs/6a4307598cd3886311591522), accessed 16 August 2026
Stated in ad	Working language is English

Responsibilities as advertised

Area	Advertised responsibilities
Platform infrastructure	• Engineer, operate, and own cloud native infrastructure on Microsoft Azure. • Provision infrastructure as code (Pulumi, Terraform, Bicep, ARM templates). • Run and optimise Kubernetes platforms (AKS) in production. • Take responsibility and ownership of the reliability, performance, security and cost of the platform. • Maintain documentation and runbooks. • Implement Security best practices such as IAM roles and network policies.
Software engineering	• Design and operate internal developer productivity platforms and tools • Develop well architected and tested code using at least one modern language • Partner with data and application teams to create platform solutions • Architect and develop scalable solutions
Kubernetes and containers	• Container Orchestration/Deployments: Capable of managing container deployment and orchestration requirements • Helm Charts: Package and deploy workloads onto Kubernetes platform • Support: Troubleshoot platform issues and support stateful services running on the platform
Monitoring and operations	• Configure and operate monitoring/observability solutions (Azure Monitor, Mimir, Grafana, Tempo) • Create logging/telemetry pipelines; decrease noise (false positives) via analysis
CI/CD and automation	• Integrate platform features with CI/CD, GitOps processes • Utilize GitHub Actions, Azure DevOps, Argo CD, Flux

Qualifications from the job ad:

- Extensive experience running cloud-native platforms in production environments
- Hands-on experience with Kubernetes and Azure
- Proven software development expertise with at least one modern programming language
- Experience architecting and building production systems, and familiarity with Infrastructure as Code tooling
- Solid understanding of platform operations, automation, and observability techniques
- Ability to develop maintainable scripts for automation (PowerShell, Python, Bash, etc.)
- Strong troubleshooting skills and experience working within a team; proficient in English

Qualified if you have: experience operating AI or ML workloads in production; familiarity with GitOps, service mesh technologies (such as Istio), or hybrid environments.

Job Advertisement 2 — Platform Engineer, Dash0

Field	Detail
Position	Platform Engineer (mid-level)
Employer	Dash0 raises \$14M Series B for OpenTelemetry-observable platform
Location	Sydney, Australia · Fully remote, though ad says engineer will need to be in Sydney area to regularly meet with local team in person
Employment type	Full-time, Engineering / R&D. 3–5 years of experience sought
Source	Ashby job board (jobs.ashbyhq.com/dash0), accessed 17 August 2026

Duties as described in job posting

- Create and maintain CI/CD pipelines/deployment automation
- Provision cloud infrastructure as code using Terraform
- Collaborate with product engineers to build scalable, cloud-native solutions
- Oversight of platform performance and availability, debugging issues when they occur
- Implement platform security best practices
- Involved with platform architecture decisions
- Create technical documentation

Qualifications they asked for

- 3-5 years experience working in platform, infrastructure, or DevOps engineering.
- Proficient in Go (GoLang)
- Experience with Terraform or similar Infrastructure as Code tools.
- Familiar with containerization and container orchestration (Docker, Kubernetes)
- Build and operate CI/CD pipelines in a cloud ecosystem (AWS, GCP, Azure).
- Excellent async communication skills.
- Present in Sydney, Australia area. Meet with team in person often.

Nice to have: Observability tooling or OpenTelemetry experience; Security compliance frameworks (SOC 2, ISO 27001) familiarity; Experience in a high-growth, venture-backed startup

1. Why did I choose these two jobs?

The intersection was what I was after. Looked at on a high level these could not be more different jobs. One's senior and wants you in the office, the other works fully remote. One was married to Azure while the other accepts AWS, GCP, or Azure. One works with industrial controls, and the other provides developers with better logging. When I saved the first and found the second about a week later however — putting the prerequisites side by side — quite a few of the tech bullets overlapped: Kubernetes, Terraform, CI/CD, logs/metrics/monitoring, some language, and security experience that stays with Platform not puts somewhere else. At that point I basically just pulled two ads as far apart as possible. Had they ended up clustered around one company or one platform I would have gone certANC, certAWS, etc. Since they don't, I'd rather practice what both want instead of learning and forgetting vendor-specific tooling later.

2. Platform engineering, what else could I have done?

Looked into it. Backend was the biggie. Cloud Engineer fell into a similar category while SysSec cropped up when I looked at jobs people took after graduating from similar lists of subjects. Those were ultimately sidelined by phrases in the job descriptions though. Job Description 1 wants you to "design and maintain internal platforms and tools to improve developer productivity" and Job Description 2 wants you to "partner with product engineering teams to design resilient, cloud-native solutions". In both cases there, your customers are other engineers. Lines up pretty neatly with the Unix elective, the sysadmin labs, how I spend my free time scripting, and even the cloud class. Backend web development isn't bad, but it isn't why I kept circling back to platform.

3. Reliability and security both fall under platform too.

Both through reading the list above and discovery afterwards. I expected security tools to be a specialist team you give things to at some point between development and monitoring. As far as the descriptions go though, neither company does that. Job Description 1 asks for you to "own platform reliability, performance, security, and cost efficiency" then later says to "apply security principles including IAM and network policies". Job Description 2 involves "embedding security best practices into platform design". Meaning Goals 7 & 8 cybersecurity and automating development tools are literally what I'm applying for when I'm done, not things I knocked out early and can stop thinking about.

4. I wanted something junior and something senior. Got two seniors instead.

Based on Job Description 2's "three to five years of professional experience" and Job Description 1 just flat out being listed as senior. Still pondering that one but ultimately settled on them both for now. Looking at entry level jobs would have netted me a list of things I need to land my first job. Useful, but only useful for the next year or so. These let me have two goals: Where I should be able to get with three to five years of experience, and where I ultimately want to go after that. Bonus points that Job Description 1 has far more detail about working with Kubernetes than Job Description 2, forcing me to actually learn about things like cost efficiency, stateful workloads, and operational runbooks I would have otherwise glossed over.

5. Remote and hybrid work affects what experience I can show.

Job Description 2 requires you to work remotely while Job Description 1 lets you work remotely half a day. They are also the first and only jobs to ask for "strong async communication skills" and "knowledge sharing through documentation". Respectively. Stopped and thought about both after finishing this paragraph. If someone who lives where my desk is right now is in their sleep schedule when I write something they'll have to be able to figure out what I meant without asking me. That extends to eventual employers reading through this page too; remote jobs can't interview me about my decisions like an in-person job could. Throw in that I'm planning on applying for these with zero production experience and it explains why each goal in Section C explicitly creates something. A repository. A runbook. A decision record. Something that if you gave it to me can explain what I was doing and why by myself.

6. Go was a guess, and observability was forced on me.

Job Description 2 explicitly asks for "solid Go (GoLang) programming skills" while mentioning OpenTelemetry under "nice to have experience with". Given that said company builds their tooling on it I'm guessing they use it somewhere internally too. Job Description 1 wants expertise in "languages commonly used for cloud native development" while mentioning Azure Monitor and three tools I know come with dashboards: Mimir, Grafana, and Tempo. As currently constructed my languages repo has examples in six languages and doesn't touch telemetry at all. Neither of which will fly. Picking Go for Goal 3 was mainly forcing myself onto something both companies list as relevant. As for monitoring services picking SLIs and SLOs treats them as something I need to do right now instead of "here's the things you should think about using".

Part B — Dream Job Skills Audit

The table below puts the three skills UTS asks of every IT graduate next to the criteria that both advertisements in Part A ask for. Each one has a **SFIA 9** code and a rating for where I am now (1 Follow, 2 Assist, 3 Apply, 4 Enable, 5 Ensure/Advise). I tried to rate against what I could actually show somebody rather than what I have read about, which pulled several ratings lower than what I first wrote down. The "level in ads" column is my own reading of the two jobs: mostly 4 to 5 for the senior role and 3 to 4 for the mid-level one. Level 3 across the core group by graduation is what looks realistic to me, with 4 in the two or three areas I already work in.

Key skill / criterion (SFIA 9 skill · code)	Current SFIA level	Level in ads	Evidence, justification and action point
Understanding of ethical principles and practices, especially in relation to Indigenous Communities (Required) <i>AI and data ethics · AIDE</i> <i>Information and data compliance · PEDP</i>	1 — Follow	3–4	Evidence: I have followed UTS academic integrity rules in every subject, and 42891 Cloud Computing raised jurisdiction and data residency as technical constraints in the first three weeks. Past that there is not much. I have not studied a professional ethics code, and I had not come across Indigenous Data Sovereignty before starting this subject. Justification: I can follow a rule once somebody gives it to me. Working out where the line sits in a case nobody has already ruled on is a different thing, and I cannot do it yet. Action point: Goal 1. Both 43030 ethics modules, the ACS Code of Professional Ethics, the Maïam nayri Wingara principles and the CARE Principles, then a written case study on a cloud region decision.
Foundational skills in Cybersecurity (Required) <i>Information security · SCTY</i> <i>Vulnerability assessment · VUAS</i> <i>Security operations · SCAD</i>	2 — Assist	4	Evidence: 48730 Cybersecurity, in progress. Lab 1 Parts 1 and 2 are submitted as a twenty-screenshot report covering information gathering, John the Ripper, SQL injection and port scanning. Week 3 was DNS pharming and PKI. I also built the isolated VM lab on Apple Silicon, which took longer than the labs themselves did. Justification: I can run a technique when I am told which one to run. Choosing the controls for a system I am responsible for is not the same task and I have not done it. Advertisement 2 asks the engineer to "embed security best practices into platform design", which is a design decision. Action point: Goal 2. Finish 48730 and the 43030 module, then put RBAC, network policies, secrets handling and image scanning onto my own cluster and write the threat model.
Foundational skills in Software Development Tools (Required) <i>Methods and tools · METL</i> <i>Configuration management · CFMG</i>	3 — Apply	4	Evidence: github.com/jhin93/stack_training — 297 commits between August 2025 and August 2026, across six languages, kept going for twelve months. I use Git for everything, coursework and self-study alike, and work in the JetBrains IDEs. Justification: Level 3 for tooling I use on my own every day. What is missing is everything that involves a second person — branch protection, pull requests, review, and any automated check that has to pass before a merge. Both ads take all of that for granted. Action point: Goal 3. A branch-and-pull-request workflow, pre-commit hooks, linting and a GitHub Actions pipeline on my own repository.
Linux and system software administration <i>System software administration · SYSP</i> <i>Systems installation and removal · HSIN</i>	3 — Apply	4	Evidence: 32547 Unix Programming, finished — ten weeks of labs and a final assignment. 32520 Network Servers Systems Administration in progress: installation and configuration, system startup, networking and services, Lab Test 1 done. I wrote the Apple Silicon installation guide that the rest of the cohort ended up using. Justification: Level 3. I can install, configure and fix Linux servers with standard procedures and without being walked through it. This is the strongest entry on the list. Action point: Use it rather than add to it. Goal 4 builds the practice cluster on Linux nodes I manage myself instead of a fully managed service, so this stays in play.
Cloud platform fundamentals (Azure, AWS, GCP) <i>Infrastructure operations · ITOP</i> <i>Infrastructure design · IFDN</i>	2 — Assist	4–5	Evidence: 42891 Cloud Computing, twelve weeks of lectures and labs: the NIST definition, IaaS/PaaS/SaaS responsibility boundaries, deployment models, hypervisors, the VM, network and storage manager layers, and SLA structure. Assignment 1 (20%) is in progress.

Key skill / criterion (SFIA 9 skill · code)	Current SFIA level	Level in ads	Evidence, justification and action point
			Justification: The theory is solid and the labs were guided end to end. What I cannot point at is anything I provisioned and then kept running for somebody else, which is the part the ads are asking about. Action point: Goal 4. Provision a managed Kubernetes cluster and run it for the full twelve weeks, including tearing it down and building it back.
Container orchestration — Kubernetes and Helm <i>Infrastructure operations · ITOP</i> <i>Systems integration and build · SINT</i>	1 — Follow	4–5	Evidence: Container concepts from 42891. No hands-on Kubernetes and no Helm. Justification: The largest single gap here. Advertisement 1 wants AKS in production and Helm packaging; Advertisement 2 wants "working knowledge of containerization and orchestration (Docker, Kubernetes)". Neither one lists it as a nice-to-have. Action point: Goal 4, Steps 1 and 3. Kubernetes fundamentals first, then a Helm-packaged two-service application. KCNA and later CKA once the hands-on part is done.
Infrastructure as Code — Terraform <i>Infrastructure design · IFDN</i> <i>Configuration management · CFMG</i>	1 — Follow	4–5	Evidence: Nothing. Every cloud resource I have created was clicked together in a console during 42891 labs. Justification: Both ads name Terraform. Advertisement 2 makes "provision and manage cloud infrastructure using Terraform" a core responsibility and Advertisement 1 adds Pulumi, Bicep and ARM templates. Console provisioning is the habit I most need to break, and it is a habit rather than a knowledge gap. Action point: Goal 4, Step 2. Rebuild the whole cluster from Terraform with remote state so that nothing exists which is not in code. Terraform Associate afterwards.
CI/CD pipelines and GitOps delivery <i>Deployment · DEPL</i> <i>Release management · RELM</i>	1 — Follow	4	Evidence: Git and GitHub daily for twelve months, but only as storage and history. No build, test or deployment pipeline has ever run against my code. Justification: "Build and maintain CI/CD pipelines and deployment automation" is the first responsibility listed in Advertisement 2, and Advertisement 1 names GitHub Actions, Azure DevOps, Argo CD and Flux. I have the version control half of this and none of the delivery half. Action point: Goal 3 for the build and test pipeline, then Goal 4, Step 4 for Argo CD, automatic sync and drift detection.
Observability, monitoring and availability <i>Availability management · AVMT</i> <i>Measurement · MEAS</i>	2 — Assist	4	Evidence: devops/server-stats.sh in my repository, written for the roadmap.sh DevOps track. It collects CPU, memory and disk use and ranks the top five processes by each. 42891 covered SLA availability targets. Justification: I can collect numbers and print them. I have never defined a service level indicator, set an objective against one, or configured an alert. Advertisement 2 wants the engineer to "monitor, troubleshoot, and improve platform performance and availability" and lists OpenTelemetry; Advertisement 1 names a whole Grafana stack. Action point: Goal 4, Step 5. Prometheus and Grafana, two SLIs with objectives against them, and alert rules I can make fire on purpose.
Programming and automation scripting <i>Programming/software development · PROG</i>	3 — Apply	4	Evidence: Working code in Go, Python, Bash, TypeScript, Java and C#: a Go project with concept notes, Python file handling and data filtering, algorithm practice, and 32998 .NET Application Development work. 32547 Unix Programming is finished. Justification: Level 3 for writing programs without supervision. The problem is shape rather than volume. Advertisement 2 asks for "solid Go (GoLang) programming skills" and Advertisement 1 for "clean, maintainable code" and "well-tested systems", and six languages with nothing tested and nothing reviewed is not that. Action point: Goal 3. Settle on Go, and add unit tests, linting and coverage to a rewritten server-stats.
Networking for distributed platforms <i>Network support · NTAS</i>	3 — Apply	4	Evidence: 49202 Communication Protocols, finished. The 32520 networking and services labs. DNS resolution and pharming analysis in 48730, written up as a case-study summary. Justification: Level 3 on real hosts. Inside a cluster I have done none of

Key skill / criterion (SFIA 9 skill · code)	Current SFIA level	Level in ads	Evidence, justification and action point
<i>Network design · NTDS</i>			it — ingress, network policy, service mesh, or load balancing for availability. Action point: Goal 4, Steps 1 and 3 cover ingress and network policies. Read up on Istio, which Advertisement 1 lists as desirable.
Incident response and problem management <i>Incident management · USUP</i> <i>Problem management · PBMG</i>	1 — Follow	4	Evidence: None in production. My troubleshooting has all been in my own lab or in coursework, where nothing is at stake and I can stop and go and read something. Justification: Advertisement 2 wants the engineer to "monitor, troubleshoot, and improve platform performance and availability"; Advertisement 1 wants runbooks and troubleshooting of stateful services. I have never worked a problem under time pressure or handed one on to somebody else. Action point: Goal 4, Step 6. Kill a node on purpose, time my own recovery, and write both the runbook and a blameless incident report.
Identity, access management and platform security posture <i>Identity and access management · IAMT</i> <i>Information security · SCTY</i>	2 — Assist	4	Evidence: PKI and certificates from 48730 Week 3, credential attacks through John the Ripper, and the shared responsibility model from 42891. Justification: I understand the mechanisms and can apply them once somebody has decided what the policy is. Advertisement 1 asks the engineer to apply IAM and network policies as the owner, which means deciding it, and the SOC 2 and ISO 27001 line in Advertisement 2 assumes the same thing. Action point: Goal 2. Least-privilege service accounts and Kubernetes RBAC on my own cluster, checked against the CIS Kubernetes Benchmark.
Cloud cost and capacity optimisation <i>Capacity management · CPMG</i>	1 — Follow	4–5	Evidence: 42891 Week 1: capital versus operating expenditure, pay-as-you-go pricing, total cost comparisons. No practical cost work at all. Justification: Theory only. Advertisement 1 lists cost efficiency among the things the engineer owns. Advertisement 2 does not mention it, which I found interesting on its own — cost ownership looks like one of the things separating the senior role from the mid-level one, so it belongs later in the plan rather than now. Action point: Goal 4, Step 6. A cost review, right-size the node pool, and write down the before and after with the reasoning.
Technical documentation and asynchronous communication <i>Specialist advice · TECH</i> <i>Stakeholder relationship management · RLMT</i>	3 — Apply	4	Evidence: Markdown installation and configuration guides for 32520 that other students used, study and mapping notes kept across subjects, the twenty-screenshot security lab report, and the fact that all of my study and assessment is done in English as a second language. Justification: Level 3 for writing to an audience I know. Advertisement 2 lists "write and maintain technical documentation" as a responsibility and "strong async communication skills" as a requirement, and Advertisement 1 wants runbooks. Remote and hybrid teams raise the bar, since the writing is doing the job that being in the room would otherwise do. Action point: Write every Goal 4 artefact for somebody in another time zone who cannot ask me a follow-up: runbooks, decision records, README files.
Self-directed professional development <i>Professional development · PDSV</i> <i>Emerging technology monitoring · EMRG</i>	3 — Apply	4	Evidence: 297 commits over twelve consecutive months of study outside coursework, a roadmap.sh DevOps path with finished projects, and separate study plans I keep for Go and Python. Justification: Level 3. I set my own learning and keep to it without a deadline forcing it, which is the main reason I think a twelve-week plan is realistic here rather than optimistic. Action point: Publish. Every Goal 4 artefact goes out publicly, because without production experience a public repository is the only claim on this page somebody else can check for themselves.

Sixteen criteria, and the split comes out at six on Level 3, four on Level 2 and six on Level 1. What I noticed writing it out is that the six Level 1 entries are not spread evenly down the list. Kubernetes, Infrastructure as Code, CI/CD and GitOps,

incident response and cost management all belong to the same activity, which is running a platform that other people depend on. That is the reason Goal 4 in Part C is one project instead of five short courses. The same cluster produces the evidence for all of them, and doing them separately would mean building most of it five times over.

Part C — Professional Learning Plan

Two areas, both taken from the audit. The first is **production platform practice**, which is the Level 1 group above — Kubernetes, Infrastructure as Code, GitOps, observability and incident response — and I have put all of it into one project as Goal 4 rather than splitting it up, for the reason at the end of Part B. The second is the **professional foundations** every UTS IT graduate needs, which are Goals 1 to 3, and which the audit shows are written into both advertisements as well. Goal 4 comes to fifty hours on its own, so I have not added a fifth goal. Each goal ends in something that can be looked at rather than a count of hours, since I will be applying without production experience and that is the only kind of claim an employer can check.

* Goal 1: Demonstrate understanding of ethical principles and practices in a professional context, especially in relation to Indigenous Communities

Consideration	Evidence or credible plan
What existing evidence do you already have (if any)?	Nothing that meets this goal. I have followed UTS academic integrity rules across every subject, and 42891 Cloud Computing treated jurisdiction and data residency as technical constraints, but I have not studied a professional ethics code and I had not come across Indigenous data governance before this subject. Current SFIA level: AIDE 1 (Follow).
What skills or knowledge do you need to gain?	1. The ACS Code of Professional Ethics, and how its six values apply to infrastructure decisions rather than to software features. 2. Indigenous Data Sovereignty — the Maïam nayri Wingara principles, and the CARE Principles for Indigenous Data Governance (Collective benefit, Authority to control, Responsibility, Ethics). 3. How the residency, replication and region choices a platform engineer makes affect who actually controls a community's data. 4. How to look at platform defaults ethically. When a platform team picks a default it is deciding for everyone who never changes it, and I had not thought about that as an ethical act. 5. Where these principles turn into obligations that can be enforced — the Australian Privacy Principles, data minimisation, cross-border disclosure — so the reasoning ends up as configuration rather than staying abstract.
What specific steps (and learning resources) will you take to fill this gap?	Step 1 (Weeks 1–3, 12 h): complete both ethics modules provided in 43030 on Canvas, including the embedded activities and the reflective questions. Step 2 (Weeks 3–4, 8 h): read and take notes on the ACS Code of Professional Ethics and Code of Conduct, and on the Australian Government AI Ethics Principles. Step 3 (Weeks 4–6, 10 h): study Indigenous Data Sovereignty through the Maïam nayri Wingara principles, the AIATSIS Code of Ethics for Aboriginal and Torres Strait Islander Research, and the UTS Jumbunna Institute resources. Step 4 (Weeks 7–9, 10 h): write an 800-word case study applying all of that to one concrete decision — choosing a cloud region and replication topology for a dataset holding Aboriginal community health records — including what I would refuse to build, and why. Step 5 (Weeks 10–11, 5 h): turn the case study into a short list of actual platform controls (region constraints, retention limits, access boundaries) written as configuration, then publish it to my portfolio. Measure of completion: both modules marked complete, and a published case study where the ethical reasoning is stated rather than implied and can be traced to specific settings.
How many hours will this take?	45 hours (12 + 8 + 10 + 10 + 5), which is the equivalent of 2 credit points.
Additional information (if required)	Target: AIDE and PEDP from Level 1 to Level 3 (Apply). I have deliberately not kept this goal separate from the technical ones. The region and replication questions it deals with are the same Terraform variables I will be setting in Goal 4, so the data residency reasoning goes into the Goal 4 architecture decision record instead of sitting in a document of its own that nobody opens.

*** Goal 2: Demonstrate foundational skills in Cybersecurity**

Consideration	Evidence or credible plan
What existing evidence do you already have?	I am not doing a Cybersecurity major, so this goal applies in full. What I have: 48730 Cybersecurity in progress, with Lab 1 Parts 1 and 2 submitted as a documented report covering information gathering, John the Ripper, SQL injection and port scanning; Week 3 work on DNS pharming and PKI; and an isolated VM lab environment I built and maintain on Apple Silicon. Current SFIA level: SCTY 2 (Assist).
What skills or knowledge do you need to gain?	1. Threat modelling with STRIDE, applied to a whole platform rather than to a single application. 2. Kubernetes security — RBAC, namespaces, network policies, pod security standards, secrets management. 3. Supply-chain security in a delivery pipeline: image vulnerability scanning, SBOM generation, signed artefacts. 4. Least-privilege identity and access management in a cloud environment, using workload identity instead of long-lived keys. 5. The compliance frameworks Advertisement 2 lists as desirable, SOC 2 and ISO 27001, and what they actually demand of access control, logging and change management.
What specific steps (and learning resources) will you take to fill this gap?	Step 1 (Weeks 1–6, 14 h): finish the remaining 48730 labs and the cybersecurity module in 43030, keeping a written log of each technique next to its defensive counterpart. Step 2 (Weeks 5–6, 8 h): work through the OWASP Top 10 and the OWASP Kubernetes Top 10, and produce a one-page STRIDE threat model for the platform built in Goal 4. Step 3 (Weeks 7–8, 10 h): harden that platform — least-privilege RBAC, namespace isolation, default-deny network policies, no plaintext secrets — using the Kubernetes hardening documentation and the CIS Kubernetes Benchmark. Step 4 (Weeks 9–10, 10 h): add Trivy image scanning as a blocking stage in the GitHub Actions pipeline from Goal 3, then document three real findings and how I fixed them. Step 5 (Weeks 11–12, 8 h): run the CIS Benchmark over the cluster, record the before and after score, and write a short gap note mapping what I have implemented against the SOC 2 and ISO 27001 control areas an employer would expect. Measure of completion: threat model published, benchmark score improved with evidence, and a pipeline that fails on a known-vulnerable image.
How many hours will this take?	50 hours (14 + 8 + 10 + 10 + 8), which is the equivalent of 2 credit points.
Additional information (if required)	Target: SCTY and IAMT from Level 2 to Level 3 (Apply). This maps onto "apply security principles including IAM and network policies" in Advertisement 1 and "embed security best practices into platform design" in Advertisement 2, which are more or less the same requirement written two different ways.

*** Goal 3: Demonstrate foundational skills in Software Development Tools**

Consideration	Evidence or credible plan
What existing evidence do you already have?	I am not doing a major in enterprise software development, so this goal applies in full. What I have: github.com/jhin93/stack_training, 297 commits between August 2025 and August 2026 across Go, Python, Bash, TypeScript, Java and C#; Git used as the record for coursework and self-study alike; the JetBrains toolchain; and a working Bash utility (server-stats.sh) written for the roadmap.sh DevOps track. Current SFIA level: METL 3 (Apply), PROG 3 (Apply).
What skills or knowledge do you need to gain?	1. Version control with other people in it — feature branches, pull requests, review, branch protection, rather than committing straight to main. 2. Automated testing and coverage in a compiled language (the Go testing package, table-driven tests). 3. Continuous integration: build, lint and test gates that have to pass before a merge (GitHub Actions). 4. Dependency, artefact and release management — Go modules, container registries, semantic versioning. 5. Static analysis and pre-commit tooling (golangci-lint, ruff, pre-commit hooks) as a normal habit rather than something I run occasionally.
What specific steps (and learning resources) will you take to fill this gap?	Step 1 (Weeks 1–2, 8 h): complete the software development tools module in 43030 and the branching material in the Pro Git book and the Atlassian Git tutorials, then restructure my repository with branch protection, a pull-request template and conventional commit messages. Step 2 (Weeks 3–5, 12 h): rewrite server-stats.sh as a tested Go CLI following the roadmap.sh project brief, using Go modules and table-driven unit tests, aiming at 70% coverage.

Consideration	Evidence or credible plan
	Step 3 (Weeks 6–7, 10 h): build a GitHub Actions workflow that runs golanci-lint, the unit tests and a container build on every pull request, and blocks the merge when it fails. Step 4 (Weeks 8–9, 8 h): publish tagged, semantically versioned container images to GitHub Container Registry from that pipeline, and add a CHANGELOG and release notes. Step 5 (Weeks 10–11, 7 h): add pre-commit hooks and static analysis across the repository, and review my own pull requests against a written checklist to get some practice at thinking like a reviewer. Measure of completion: nothing reaches main without passing CI, and there is a green build badge and a released, versioned artefact visible on the public repository.
How many hours will this take?	45 hours (8 + 12 + 10 + 8 + 7), which is the equivalent of 2 credit points.
Additional information (if required)	Target: METL and CFMG from Level 3 to Level 4 (Enable), with PROG staying at Level 3 but concentrated in Go. Advertisement 2 asks for "solid Go (GoLang) programming skills" and Advertisement 1 for "clean, maintainable code in at least one modern programming language" and "scalable, well-tested systems". Narrowing six languages down to one is the least comfortable decision in this plan, but spreading myself across all of them has not produced anything I would be willing to show an employer.

Goal 4: Design, build and operate a production-style Kubernetes platform that is provisioned entirely as code, delivered by GitOps, and observable — the core skill shared by both target job advertisements

Consideration	Evidence or credible plan
What existing evidence do you already have?	Foundations, but not the skill itself. 42891 Cloud Computing (twelve weeks: NIST model, IaaS/PaaS/SaaS boundaries, hypervisors, the VM, network and storage manager layers, SLAs; Assignment 1 in progress); 32520 Network Servers Systems Administration (installation, configuration, startup, networking and services, Lab Test 1); 32547 Unix Programming and 49202 Communication Protocols both finished; and a Bash metrics collector in my repository. Current SFIA levels: ITOP 1–2, IFDN 1, DEPL 1, AVMT 2, CPMG 1, USUP 1.
What skills or knowledge do you need to gain?	1. Kubernetes: core objects, scheduling, ingress, stateful workloads, and Helm packaging with values-driven configuration. 2. Infrastructure as Code: Terraform with remote state, modules, and a plan/apply lifecycle that is actually reproducible (Advertisement 1 also names Pulumi, Bicep and ARM templates). 3. GitOps delivery with Argo CD or Flux — a repository of record, automatic sync, drift detection. 4. Observability and reliability: Prometheus, Grafana and OpenTelemetry, plus the discipline of defining indicators and objectives rather than only collecting metrics. 5. Operations under pressure: incident response, writing runbooks, and treating capacity and cost as ongoing engineering work.
What specific steps (and learning resources) will you take to fill this gap?	Step 1 — Kubernetes fundamentals (Weeks 1–2, 8 h): work through the kubernetes.io tutorials and the KCNA syllabus, and stand up a three-node k3s cluster on Linux VMs, reusing the 32520 skills. Measure: pod, deployment, service and ingress manifests committed, and a workload reachable from outside the cluster. Step 2 — Infrastructure as Code (Weeks 3–4, 10 h): complete the HashiCorp Developer Terraform Associate tutorials, then provision a managed cluster entirely from Terraform with remote state, on student cloud credit. Measure: destroy and re-apply from a clean state gives back the same platform with no manual step in between. Step 3 — Packaging (Weeks 5–6, 8 h): Helm-package a two-service application (the Go CLI service from Goal 3 plus PostgreSQL) with separate dev and prod values. Measure: one helm install command puts the whole stack into a clean namespace. Step 4 — GitOps delivery (Weeks 7–8, 10 h): install Argo CD, make the Git repository the single source of truth, and wire it to the Goal 3 pipeline. Measure: a merge to main reaches the cluster inside five minutes with no manual kubectl apply, and a change I make out of band gets picked up as drift and reverted. Step 5 — Observability (Weeks 9–10, 8 h): deploy kube-prometheus-stack, instrument the Go service with OpenTelemetry, build a Grafana dashboard, and define two indicators (availability and p95 latency) with objectives and alert rules, working from the Google SRE Workbook chapters on SLOs and alerting. Measure: an induced failure fires the alert and shows up on the dashboard. Step 6 — Reliability and cost (Weeks 11–12, 6 h): run a chaos exercise by terminating a node, time my own recovery, and write the runbook and a blameless incident report. Then do a cost review, right-size the node pool, and record the change. Measure: runbook, incident report, and

Consideration	Evidence or credible plan
	a documented before and after monthly cost.
How many hours will this take?	50 hours (8 + 10 + 8 + 10 + 8 + 6), the equivalent of 2 credit points. Goal 4 reaches the full 40–50 hour requirement on its own, so a fifth goal is not needed.
Additional information (if required)	On completion: ITOP, IFDN and DEPL from Level 1 to Level 3 (Apply), AVMT from 2 to 3, with Level 4 as the target after graduation through an associate infrastructure role. Certifications come after the hands-on work, in the order KCNA, Terraform Associate, CKA, so that the certificate is describing something I have done rather than standing in for it. Everything from Steps 1 to 6 goes into one public repository — Terraform modules, the Helm chart, the Argo CD configuration, the Grafana dashboard JSON, the runbook, the incident report and the cost analysis. Both advertisements are remote or hybrid and both assume the engineer can be trusted to work and write without somebody watching, and I will be applying without production experience, so the repository has to do the job a reference check would normally do.

Sources and Learning Resources

Reference list — not part of the four-page Professional Learning Plan.

Kongsberg Digital (Falkor), Senior Platform Engineer, Fornebu. The Hub. <https://thehub.io/jobs/6a4307598cd3886311591522> (accessed 16 August 2026).

Dash0, Platform Engineer, Sydney, Australia. Ashby job board. <https://jobs.ashbyhq.com/dash0/c71c1a08-d9ea-4229-b7c9-a3ac9eabc95c> (accessed 17 August 2026).

SFIA Foundation, SFIA 9 — Skills A–Z and Levels of Responsibility. <https://sfia-online.org/en/sfia-9> (accessed 16 August 2026).

Australian Computer Society, ACS Code of Professional Ethics and Code of Conduct.

Maia nayri Wingara Indigenous Data Sovereignty Collective, Indigenous Data Sovereignty Principles.

Global Indigenous Data Alliance, CARE Principles for Indigenous Data Governance.

AIATSIS, Code of Ethics for Aboriginal and Torres Strait Islander Research.

Office of the Australian Information Commissioner, Australian Privacy Principles.

Beyer, B. et al., The Site Reliability Workbook, Google (chapters on SLOs and alerting).

Center for Internet Security, CIS Kubernetes Benchmark; OWASP Kubernetes Top 10.

HashiCorp Developer, Terraform Associate tutorials; kubernetes.io tutorials; Argo CD documentation.

Own evidence base: github.com/jhin93/stack_training — 297 commits, August 2025 to August 2026.